Séance 4 – TP

Réalisation d'algorithmes

Objectifs de la séance

- Traduire un algorithme en Python avec variables, conditions et boucles
- Implémenter des algorithmes classiques : Fibonacci, PGCD, années bissextiles, nombres premiers
- Modéliser et simuler des **automates à états** : feu tricolore, distributeur, reconnaissance de chaînes
- Tester systématiquement son code sur des cas connus

⚡ Quiz flash : séance 3

1. Que vaut `"ERROR" in "2026-10-21 ERROR disk full"` ?
2. Corriger : `if level == "ERROR" or "FATAL":`
3. Quand le `else` d'une boucle `for` s'exécute-t-il ?
4. Qu'affiche `for i, s in enumerate(["a", "b"], start=1): print(i, s)` ?
5. Écrire en une ligne : `msg` vaut `"pair"` si `n` est pair, sinon `"impair"`.

Déroulé du TP

Durée	Bloc
10 min	Quiz flash
10 min	Méthode + rappels sur les algorithmes numériques
40 min	🧪 Partie 1 : Fibonacci, PGCD, bissextiles, nombres premiers
5 min	Pause
5 min	Les automates à états
45 min	🧪 Partie 2 : feu tricolore, distributeur, reconnaissance de chaînes, synthèse

Méthode de travail

1. **Comprendre** : reformuler, choisir des exemples dont on connaît le résultat
2. **Écrire l'algorithme** en français ou en pseudo-code
3. **Traduire** en Python, par petites étapes
4. **Tester** : cas normal, cas limites (0, 1, valeurs négatives, liste vide)
5. **Relire** : noms explicites, pas de code dupliqué

💡 Les cellules ✅ du notebook jouent le rôle de **tests** : en S15, nous écrirons de vrais tests unitaires.

Rappels : quatre algorithmes classiques

Algorithme	Idée clé
Fibonacci	`a, b = b, a + b` à chaque tour, à partir de 0 et 1
PGCD (Euclide)	`pgcd(a, b) = pgcd(b, a % b)` jusqu'à ce que `b == 0`
Année bissextile	divisible par 4 **et** non divisible par 100, **ou** divisible par 400
Nombre premier	aucun diviseur entre 2 et √n (tester `d * d <= n`)

```
a, b = 1071, 462
while b != 0:
    a, b = b, a % b
print(a)    # 21
```

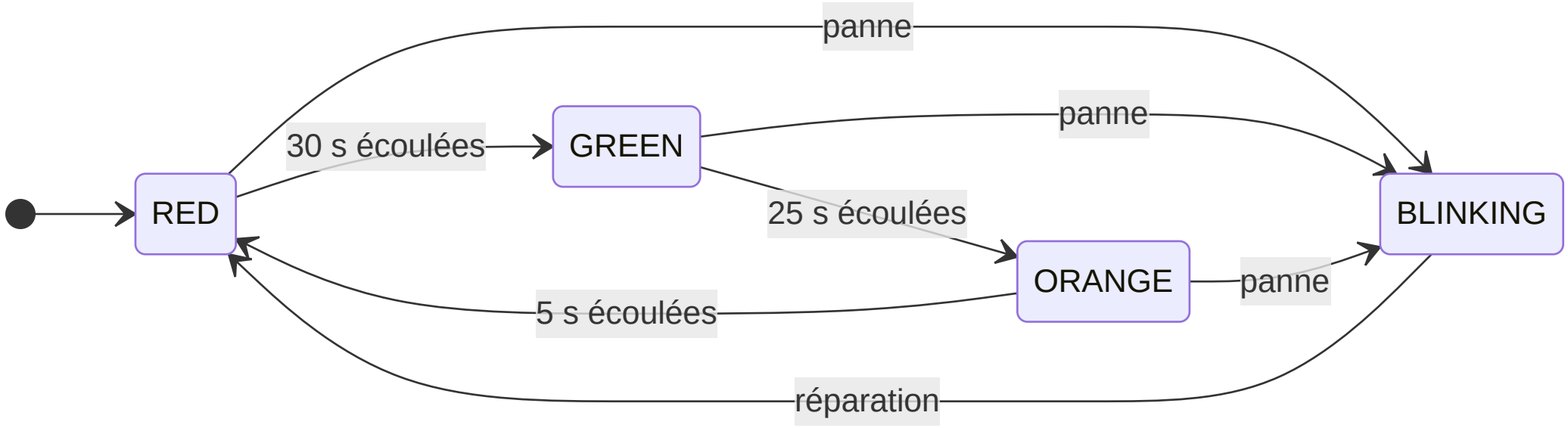
🧪 À vous : partie 1 (40 min)

Notebook `S04_lab_enonce.ipynb`, partie 1

- Ex. 1 ⭐ : les n premiers termes de Fibonacci
- Ex. 2 ⭐⭐ : premier terme de Fibonacci qui dépasse un seuil
- Ex. 3 ⭐ / ⭐⭐ : PGCD et PPCM (planification de tâches périodiques)
- Ex. 4 ⭐ : années bissextiles entre 1900 et 2030
- Ex. 5 ⭐⭐ : un nombre est-il premier ?
- Ex. 6 ⭐⭐⭐ : crible d'Ératosthène

Les automates à états

Un **automate** = un ensemble d'**états**, un état initial, et des **transitions** déclenchées par des **événements**.



En Python : l'état courant dans une variable, les transitions dans un **dictionnaire**.

Table de transitions avec un dictionnaire

```
transitions = {
    "RED":    ("GREEN", 30),     # état suivant, durée de l'état courant
    "GREEN":  ("ORANGE", 25),
    "ORANGE": ("RED", 5),
}
state = "RED"
elapsed = 0
for step in range(4):
    next_state, duration = transitions[state]
    print(f"t={elapsed:>3}s : {state} pendant {duration}s")
    elapsed += duration
    state = next_state
```

```
t=  0s : RED pendant 30s
t= 30s : GREEN pendant 25s
t= 55s : ORANGE pendant 5s
t= 60s : RED pendant 30s
```

Reconnaître une chaîne avec un automate

Format d'un numéro de ticket : `INC-` suivi de **4 chiffres** (ex. `INC-2048`).

État	Lu	Nouvel état
START	`I`	I
I	`N`	N
N	`C`	C
C	`-`	DASH
DASH, D1, D2, D3	chiffre	D1, D2, D3, D4
tout autre cas		ERROR

La chaîne est **acceptée** si l'automate finit dans l'état `D4`.

🧪 À vous : partie 2 (45 min)

Notebook `S04_lab_enonce.ipynb`, partie 2

- Ex. 7 ⭐⭐ : simulation d'un feu tricolore avec mode clignotant
- Ex. 8 ⭐⭐⭐ : distributeur de boissons (monnaie, rendu)
- Ex. 9 ⭐⭐ : nombres binaires divisibles par 3
- Ex. 10 ⭐⭐⭐ : reconnaissance des numéros de ticket
- Synthèse : workflow d'un ticket d'incident
- Bonus GestParc : valider le format des numéros d'inventaire

Bilan du TP

- Un algorithme se **teste** sur des cas connus et des cas limites
- Le motif `a, b = b, a + b` évite les variables temporaires
- La boucle `while` convient quand on ne connaît pas le nombre de tours (Euclide, seuil)
- Un automate = **état courant + table de transitions** (dictionnaire)
- Le code de cette séance contient beaucoup de répétitions…

Séance 5 : les fonctions, pour rendre ces algorithmes réutilisables